WEEK 6 - PYTHON PROGRAMMING

Files, Classes and Charts

Your program has amnesia. Today we cure it.

ACT 1 · FILES
remember

ACT 2 · CLASSES
structure

ACT 3 · CHARTS
convince

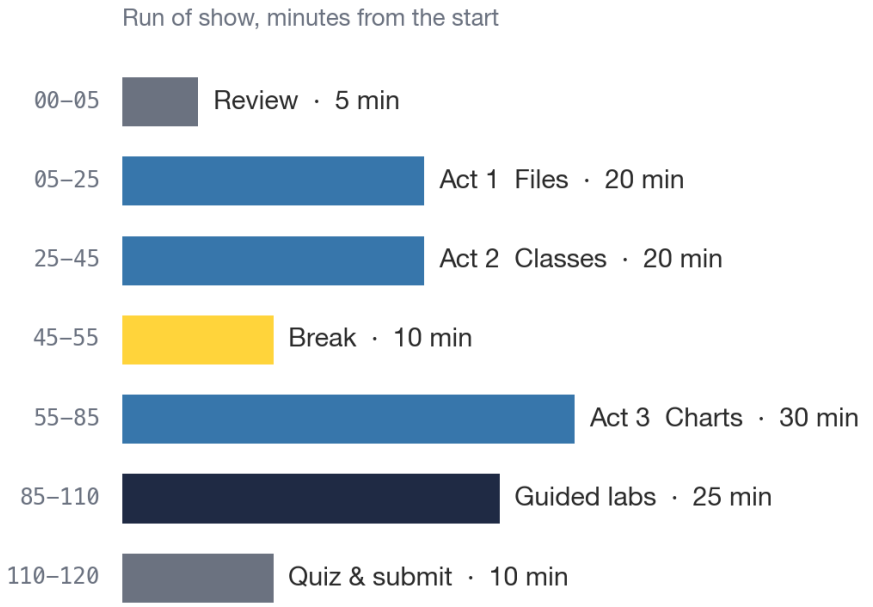
120 minutes, including a break

Instructor: MinGi KYUNG

The Plan: Three Superpowers, One Story

Friday evening. The trading week is over. Your program knows everything, and forgets it all at exit.

- Memory: open() and with. Your report survives the program that wrote it.
- Identity: class. A Position that knows its own profit.
- Persuasion: matplotlib. Numbers become lines, bars and histograms.
- One story the whole way: the trading week, saved, structured, drawn.



Review: Where Did the Numbers Go?

- Last session a Series answered every question with one call.
- price_report.py is due now. It prints four numbers.
- Then the program ends, and the four numbers are gone.
- Today: save them, structure them, draw them.

PREDICT Run it again tomorrow. Where is yesterday's answer?

PYTHON

```
import pandas as pd

closes = pd.Series([10000, 10200, 9800, 10500,
                    10800])
print(closes.mean())   # one question, one answer
```

OUTPUT

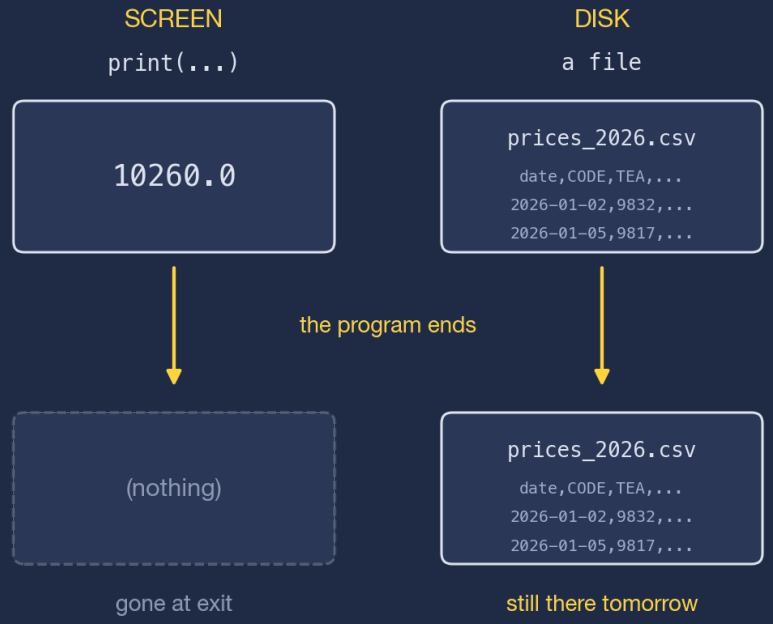
```
10260.0
program ends. Screen clears. Answer: gone.
```

ACT 1 · FILES

Teach Your Program to Remember

print() talks. A file remembers.

- Everything you wrote so far forgets everything at exit.
- The screen is not storage. The disk is.
- A file is your program's memory: written today, still there tomorrow.
- Plot twist: you have used files all along. prices_2026.csv is one.



open(): Three Moves and One Trap

- open(path, "w") creates the file and hands you a file object.
- f.write(text) puts a string in the file. Nothing more.
- f.close() hands the file back to the system.
- Skip close() and the file may stay empty or locked.

TRAP write() adds no newline. End your lines with "\n" yourself.

PYTHON

```
report = open("summary.txt", "w")
report.write("CODE week high: 10800")
report.write("\n")
report.close()
print("Saved summary.txt")
```

OUTPUT

```
Saved summary.txt
summary.txt: one line, CODE week high: 10800
```

with: The Door That Closes Itself

- with open(...) as f: closes the file when the block ends.
- An error inside the block? Still closed on the way out.
- No forgotten close(), no locked file, no half-written report.
- From this slide on, every open() wears a with.

PREDICT Run this script twice. How many lines does summary.txt hold? Answer on slide 9.

PYTHON

```
lines = ["CODE 10800", "TEA 4800"]
with open("summary.txt", "w") as f:
    for line in lines:
        f.write(line + "\n")
print("Saved summary.txt")
```

OUTPUT

```
Saved summary.txt
summary.txt: 2 lines, each ending in \n
```

Reading: Three Ways to Drink from a File

- read(): the whole file as one string. Drink the glass.
- readlines(): a list of lines. Pour it into cups.
- for line in f: one line at a time. Sip. Memory-friendly.
- No mode means "r". Reading is the default.

TRAP Every line ends with "\n". strip() takes it off. Without it, "buy\n" is not "buy".

PYTHON

```
with open("trades.csv") as f:
    header = f.readline()
    print(header.strip())      # date,ticker,...

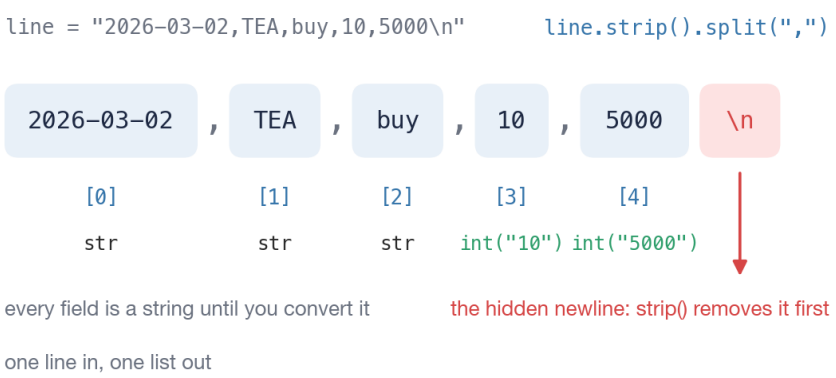
with open("trades.csv") as f:
    print(len(f.readlines()))  # 11 with header
```

OUTPUT

```
date,ticker,action,shares,price
11
10 trades + 1 header line = 11
```

Anatomy of a CSV Line

- strip() removes the "\n" hiding at the end.
- split(",") cuts the line into a list of fields.
- Fields are strings, even "3". int() makes it a number.
- Congratulations: you just did what read_csv does.



PYTHON

```
with open("trades.csv") as f:
    f.readline()
    line = f.readline()

fields = line.strip().split(",")
print(fields[1], fields[2],
      int(fields[3]) * int(fields[4]))
```

OUTPUT

```
TEA buy 50000
the first trade of the week: 10 shares x 5000
```

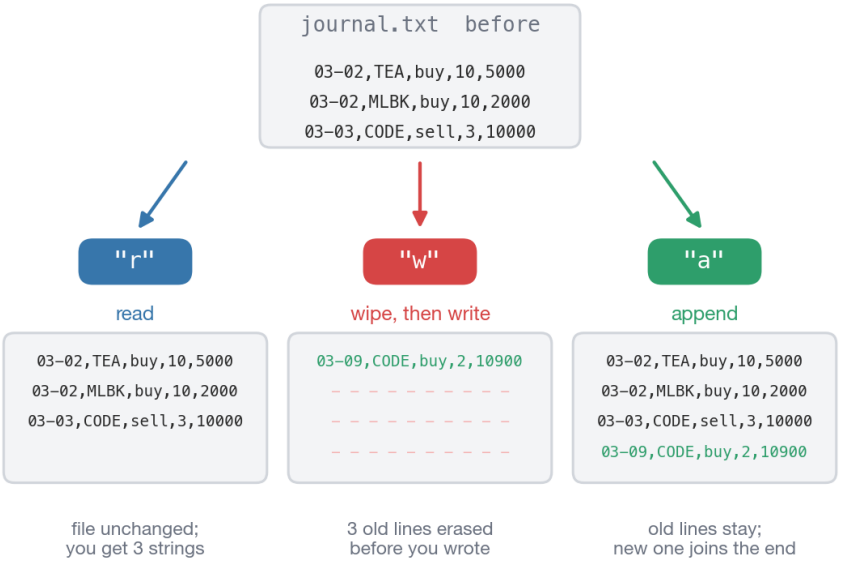
Modes: r Reads, w Wipes, a Adds

- "r" reads. Missing file? FileNotFoundError, loud and clear.
- "w" writes from scratch. It erases the file first. Every time.
- "a" appends: old lines stay, new lines join the end.
- Reports that replace themselves: "w". Journals that grow: "a".

PAYOFF Slide 6, run twice: still 2 lines. "w" wiped run one before run two wrote.

PYTHON

```
with open("journal.txt", "a") as f:
    f.write("2026-03-09,CODE,buy,2,10900\n")
```



Practice 1: Be the Parser

For five minutes, you are pd.read_csv.

- Open journal_by_hand.py. trades.csv sits beside it.
- The starter opens the file and skips the header for you.
- Read every line. strip, split, look at the action field.
- Count "buy" and "sell". Print both counts. 5 minutes.

TIP The action is fields[2]. Two counters are all you need.

OUTPUT

```
buy 6
sell 4
```



The Payoff: Twelve Lines Become One

You just parsed a file by hand. Here is what that was worth.

- pd.read_csv does strip, split and int() for every line.
- Columns arrive named. Numbers arrive as numbers.
- Same question, same answer: 6 buys.
- After the break, the year file loads the exact same way.

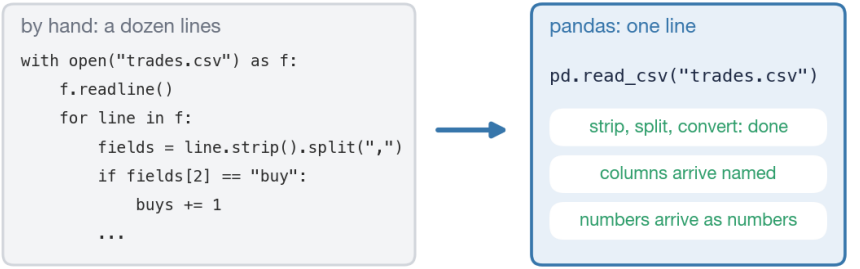
PYTHON

```
import pandas as pd

log = pd.read_csv("trades.csv")
print(log["action"].value_counts()["buy"])  # 6
```

OUTPUT

```
6
the same 6 your counters printed. One line.
```

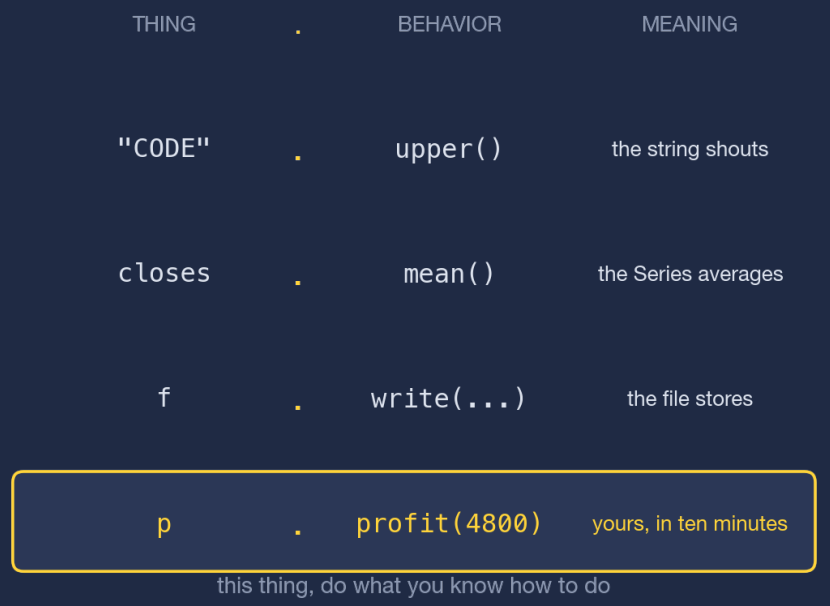


ACT 2 · CLASSES

You Already Talk to Objects

The dot is a sentence: this thing, do what you know.

- "CODE".upper(): the string knows how to shout.
- closes.mean(): the Series knows how to average.
- f.write(...): the file knows how to store.
- An object is data plus the behavior that fits it. Today you design one.

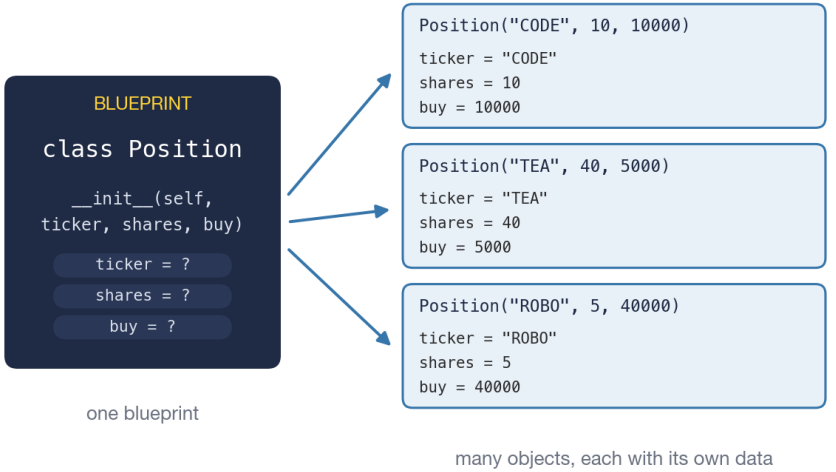


Blueprint and Buildings: class Position

- class Position: draws the blueprint for a new kind of object.
- __init__ runs every time a Position is built.
- self is the object under construction. Always the first parameter.
- Everything you assign to self becomes that object's own data.

PYTHON

```
class Position:
    def __init__(self, ticker, shares, buy):
        self.ticker = ticker
        self.shares = shares
        self.buy = buy
```



one blueprint

many objects, each with its own data

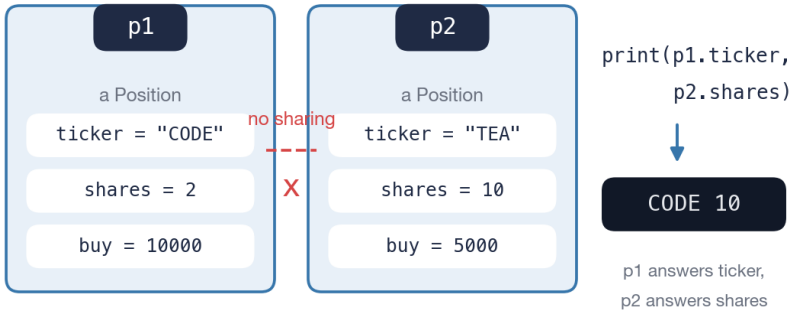
Attributes: Every Object Packs Its Own Bag

- Attributes are the data inside one object.
- p1 and p2 come from one class but share nothing.
- p1.ticker is CODE. p2.ticker is TEA. They never mix.
- Reach an attribute with a dot: object.name.

PYTHON

```
class Position:
    def __init__(self, ticker, shares, buy):
        self.ticker = ticker
        self.shares = shares
        self.buy = buy

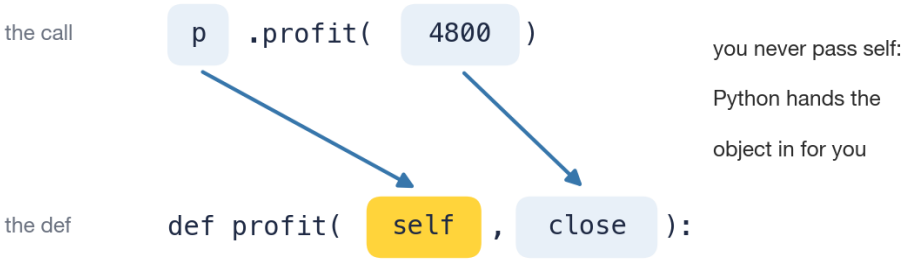
p1 = Position("CODE", 2, 10000)
p2 = Position("TEA", 10, 5000)
print(p1.ticker, p2.shares)   # CODE 10
```



Methods: Teach the Object a Trick

- A method is a function inside the class.
- self is first in the def, never in the call.
- profit(self, close) uses self.shares and self.buy.
- p.profit(4800): Python passes p in as self.

TRAP No self in the def? Python says: takes 1 argument but 2 were given.



PYTHON

```
class Position:
    def __init__(self, ticker, shares, buy):
        self.ticker = ticker
        self.shares = shares
        self.buy = buy

    def profit(self, close):
        return (close - self.buy) * self.shares

print(Position("TEA", 40, 5000).profit(4800))
```

OUTPUT

```
-8000
40 shares, bought at 5000, now 4800. Ouch.
```

One Loop, Five Objects, Two Destinations

- The portfolio is a list of Position objects.
- Each object carries its own numbers in its own bag.
- One loop asks every position the same question.
- The answer goes to the screen and the file in the same breath.

OUTPUT

```
CODE 10
TEA 40
ROBO 5
report.txt holds the same three lines
```

PYTHON

```
class Position:
    def __init__(self, ticker, shares, buy):
        self.ticker = ticker
        self.shares = shares
        self.buy = buy

portfolio = [Position("CODE", 10, 10000),
             Position("TEA", 40, 5000),
             Position("ROBO", 5, 40000)]

with open("report.txt", "w") as f:
    for p in portfolio:
        line = p.ticker + " " + str(p.shares)
        f.write(line + "\n")
        print(line)
```

Practice 2: The Position Report

- Open position_report.py.
- Finish the class: add profit(self, close).
- Build all five positions from the starter's rows.
- Print each ticker and profit. Save it all to portfolio_report.txt. 5 minutes.

TIP The starter has the rows, Friday's closes and the with block. You write the method and the loop.

OUTPUT

```
CODE +8000
TEA -8000
ROBO +25000
MLBK +15000
FIT +24000
Saved portfolio_report.txt
four winners, one loser. TEA, we need to talk.
```

BREAK

Halftime

10 minutes. Stretch, refill, come back with answers.

- What does with open(...) as f: guarantee?
- What is self inside a method?
- Why did Practice 1 skip the first line?

> **STORY** Coffee is opened in mode "a". It only ever accumulates.

Next: the numbers become pictures.

ACT 3 · CHARTS

Numbers That Persuade

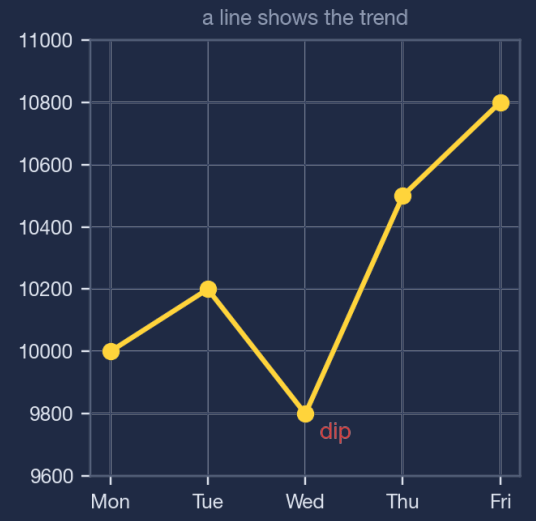
Files remember. Objects structure. Charts convince.

- A table makes you compute. A line shows the trend.
- matplotlib is Python's standard plotting library.
- NumPy computes, pandas labels, matplotlib draws.
- Same five closes. Which side shows Wednesday's dip faster?

a table asks you to compute

Mon	10,000
Tue	10,200
Wed	9,800
Thu	10,500
Fri	10,800

a line shows the trend



One Install, One Alias: plt

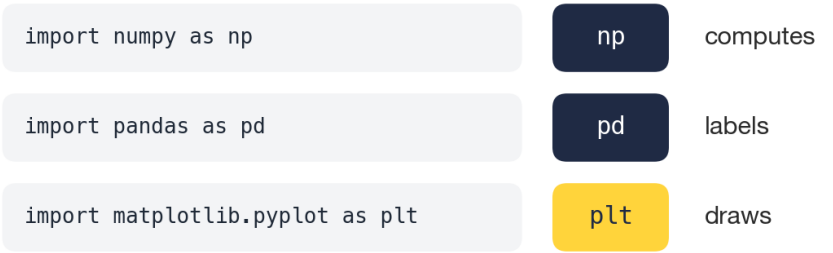
- Locally: pip install matplotlib, in the terminal.
- Colab ships matplotlib already. Skip the install.
- The convention: import matplotlib.pyplot as plt.
- np, pd, plt: the three-letter club. The whole world writes it this way.

OUTPUT

```
$ pip install matplotlib
Successfully installed matplotlib-3.9.4 ...
```

PYTHON

```
import matplotlib.pyplot as plt   # like np, pd
```



the three-letter club: the whole world writes it this way

Your First Line Plot

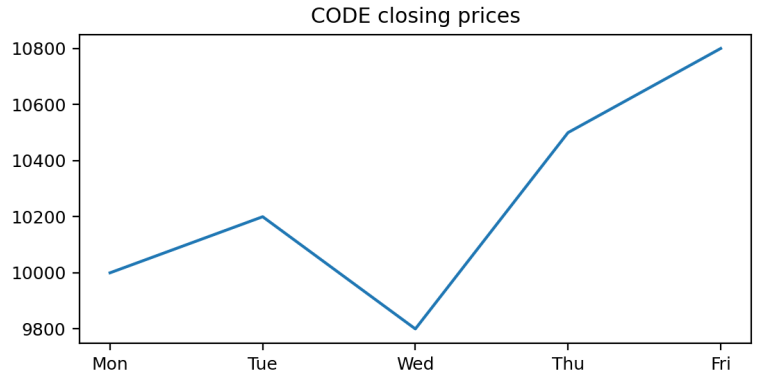
- plt.plot(x, y) draws one line, point to point.
- x first: the days. y second: the closes.
- The figure is the canvas. The axes is the chart on it.
- savefig writes the chart to a file. show() opens it in class.

PYTHON

```
import matplotlib.pyplot as plt

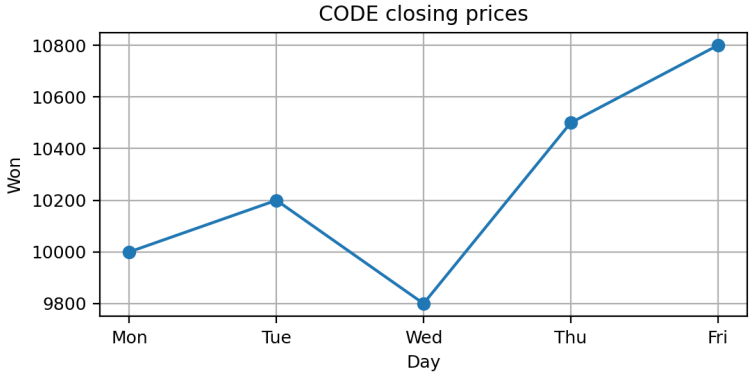
days = ["Mon", "Tue", "Wed", "Thu", "Fri"]
closes = [10000, 10200, 9800, 10500, 10800]

plt.plot(days, closes)
plt.title("CODE closing prices")
plt.savefig("week.png")
# plt.show() opens the chart in class
```



Titles, Labels, Grid, Markers

- title names the chart. xlabel and ylabel name the axes.
- grid(True) adds reading lines. marker="o" marks each day.
- A labeled chart needs no explanation.
- An unlabeled chart needs an apology.



PYTHON

```
import matplotlib.pyplot as plt

days = ["Mon", "Tue", "Wed", "Thu", "Fri"]
closes = [10000, 10200, 9800, 10500, 10800]

plt.plot(days, closes, marker="o")
plt.title("CODE closing prices")
plt.xlabel("Day")
plt.ylabel("Won")
plt.grid(True)
plt.savefig("labeled.png")
```

Two Lines and a Legend

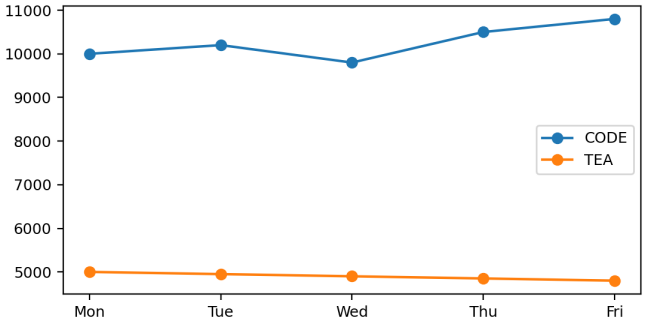
- Call plot once per line. They share one chart.
- label= names each line. legend() shows the names.
- CODE climbs while TEA slides: visible, not printed.
- linestyle="--" and color= restyle when the story needs it.

PYTHON

```
import matplotlib.pyplot as plt

days = ["Mon", "Tue", "Wed", "Thu", "Fri"]
code = [10000, 10200, 9800, 10500, 10800]
tea = [5000, 4950, 4900, 4850, 4800]

plt.plot(days, code, label="CODE", marker="o")
plt.plot(days, tea, label="TEA", marker="o")
plt.legend()
plt.savefig("two.png")
```



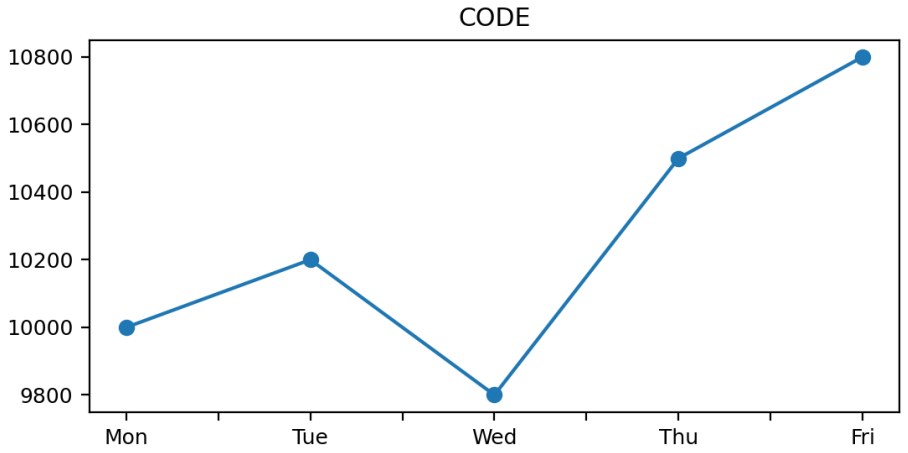
The pandas Shortcut

- A Series can draw itself: closes.plot().
- The index becomes the x axis automatically.
- title= is a plot argument here.
- pyplot still saves: plt.savefig after.

PYTHON

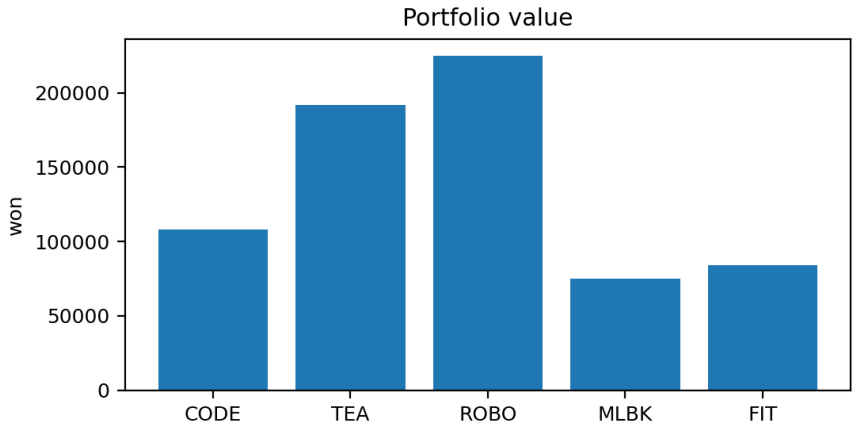
```
import pandas as pd
import matplotlib.pyplot as plt

closes = pd.Series([10000, 10200, 9800,
                    10500, 10800],
                   index=["Mon", "Tue", "Wed",
                          "Thu", "Fri"])
closes.plot(title="CODE", marker="o")
plt.savefig("series.png")
```



Bar Charts: Who Is Biggest?

- plt.bar(x, y) draws one bar per category.
- Lines show change. Bars compare size.
- Your five Position values, one bar each.
- ROBO's lead is visible before anyone prints a number.



PYTHON

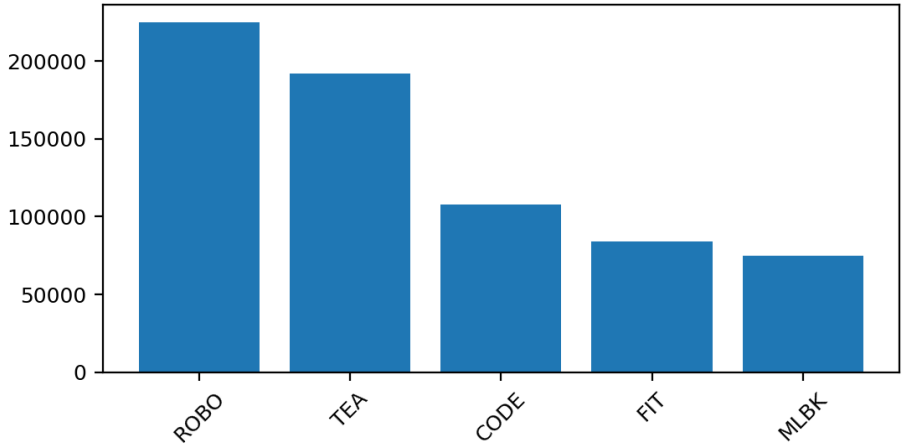
```
import matplotlib.pyplot as plt

tickers = ["CODE", "TEA", "ROBO",
           "MLBK", "FIT"]
values = [108000, 192000, 225000,
          75000, 84000]

plt.bar(tickers, values)
plt.title("Portfolio value")
plt.ylabel("won")
plt.savefig("values.png")
```

Sort, Then Bar

- Sorted bars read instantly: biggest first.
- sorted(zip(values, tickers)) pairs, then ranks.
- xticks(rotation=45) slants long labels.
- figsize=(w, h) sets the canvas size.



PYTHON

```
import matplotlib.pyplot as plt

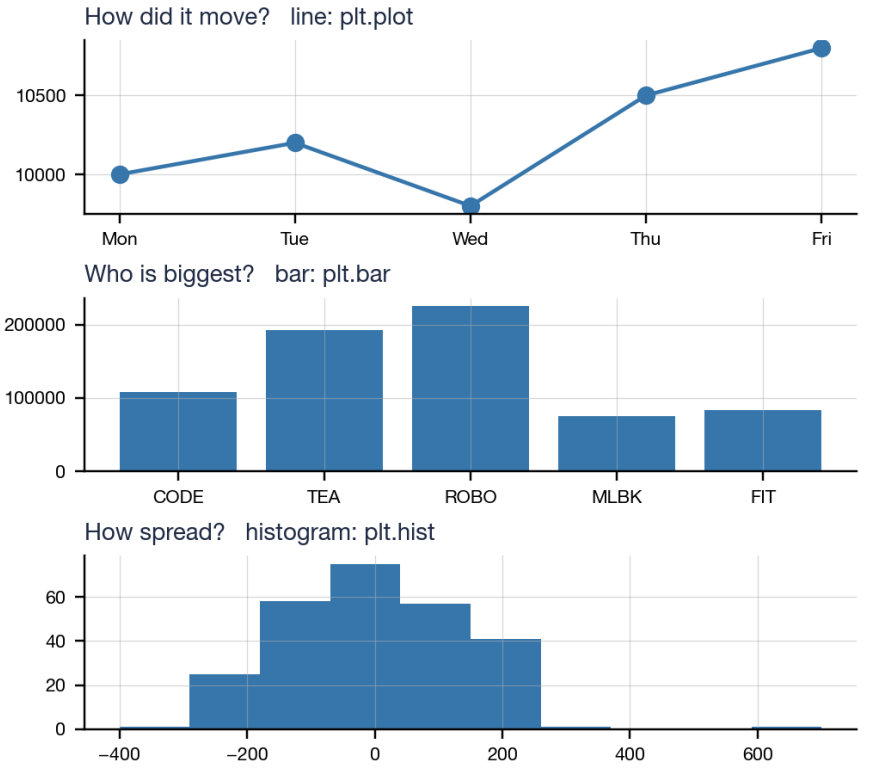
tickers = ["CODE", "TEA", "ROBO",
           "MLBK", "FIT"]
values = [108000, 192000, 225000,
          75000, 84000]

pairs = sorted(zip(values, tickers),
               reverse=True)
plt.bar([t for v, t in pairs],
        [v for v, t in pairs])
plt.xticks(rotation=45)
plt.savefig("ranked.png")
```

Choosing the Chart

The question picks the chart. Not your taste.

- How did it move over time? Line: plt.plot.
- Who is biggest? Bar: plt.bar.
- How spread are the values? Histogram: plt.hist.
- Wrong chart, wrong answer, even with the right data.



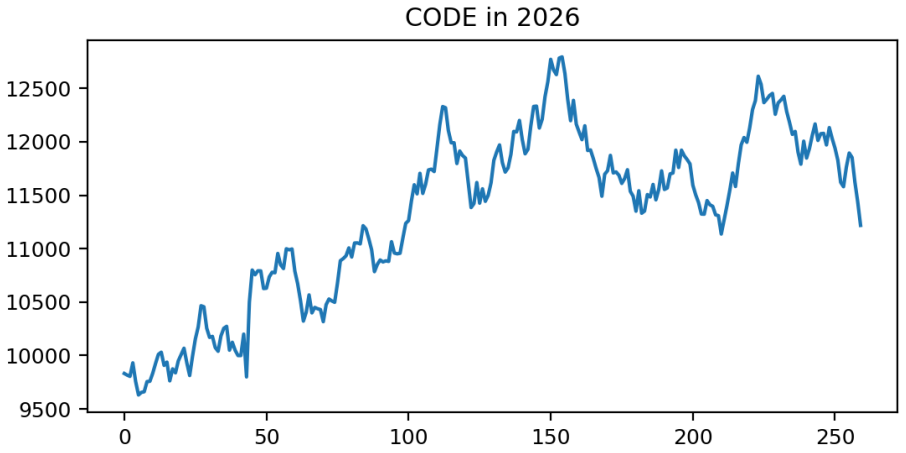
The Year from Disk

- This morning you parsed trades.csv by hand.
- read_csv loads the whole year the same way.
- Plot one column and 260 trading days appear.
- One file, one line of code, one year.

PYTHON

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("prices_2026.csv")
df["CODE"].plot(title="CODE in 2026")
plt.savefig("code.png")
print(len(df))   # 260
```



Histograms: How Spread?

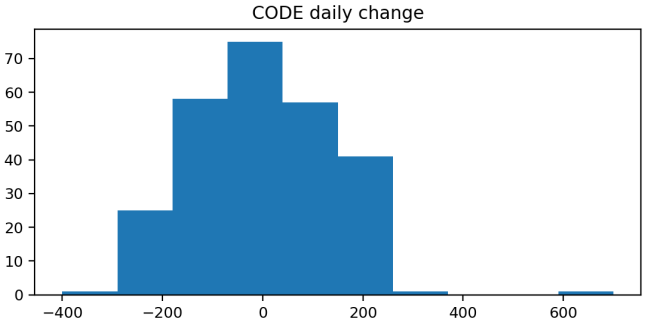
- diff() gives each day's change from the day before.
- dropna() drops day one, which has no yesterday.
- plt.hist counts the changes into bins.
- The shape is the risk story: narrow is calm, wide is wild.

PYTHON

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("prices_2026.csv")
changes = df["CODE"].diff().dropna()

plt.hist(changes)
plt.title("CODE daily change")
plt.savefig("changes.png")
print(len(changes))   # 259
```



Saving and Fresh Charts

- savefig writes the current figure to a file.
- png files paste into reports and slides.
- figure() starts a fresh, empty chart.
- Two charts in one script need one figure() between them.

TRAP No figure()? The bars land on top of the line. One file, one mess.

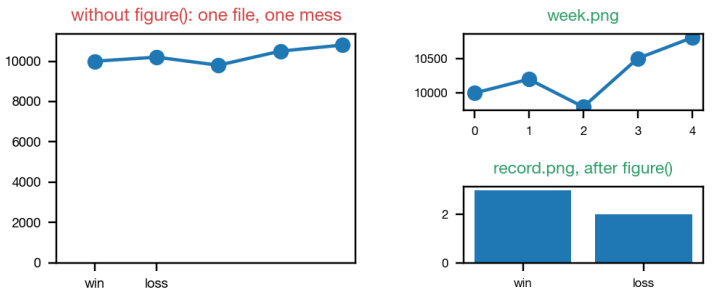
PYTHON

```
import matplotlib.pyplot as plt

closes = [10000, 10200, 9800, 10500, 10800]

plt.plot(closes, marker="o")
plt.savefig("week.png")   # file for reports

plt.figure()              # a fresh chart
plt.bar(["win", "loss"], [3, 2])
plt.savefig("record.png")
```



Lab 1: Friday Evening Charts

- Open my_charts.py.
- Chart 1: CODE's week as a line, with markers and a grid.
- Chart 2: portfolio values as bars.
- Print the week's high and both saved file names. 9 minutes.

TIP Remember figure() between the two charts. One wall, two rooms.

OUTPUT

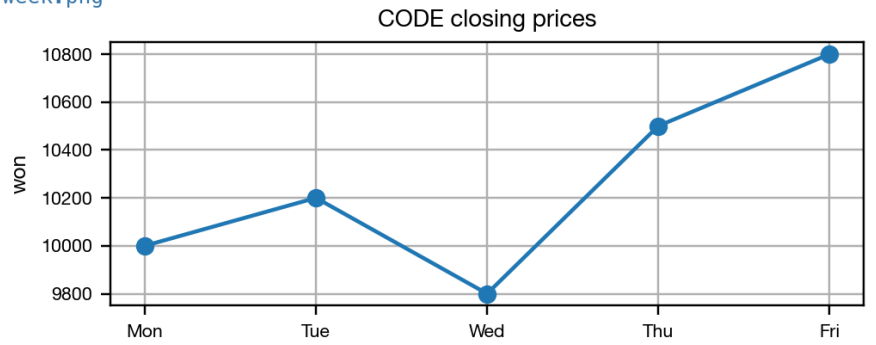
```
High: 10800
Saved week.png
Saved values.png
```

Lab 1 Checks

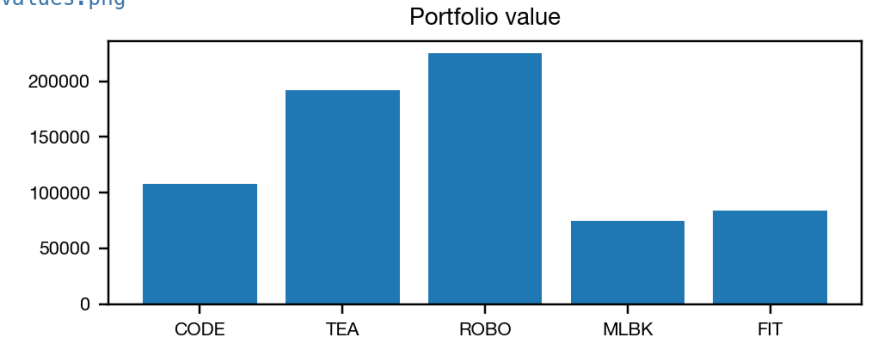
- week.png: five points, one line, labeled and gridded.
- values.png: five bars, titled, y axis in won.
- Two separate files, not one layered chart.
- The printed high matches max(closes): 10800.

> **TIP** The y axis starts near 9800, not 0. matplotlib zooms to the data.

`week.png`



`values.png`



Lab 2: The Trading Year

- Open year_report.py. prices_2026.csv sits beside it.
- Chart 1: CODE's year as a line, titled.
- Chart 2: a histogram of CODE's daily changes.
- Print the day count, the best close and its date. 11 minutes.

TIP idxmax() gives the row of the best close. df["date"] turns that row into a date.

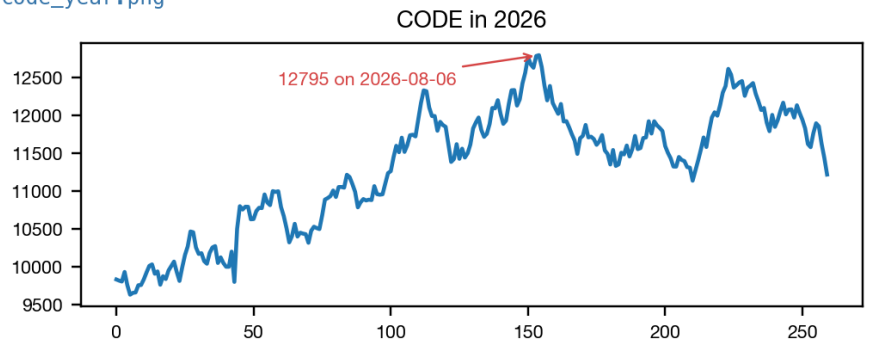
OUTPUT

```
Days: 260
Best close: 12795
Best date: 2026-08-06
Saved code_year.png
Saved changes.png
```

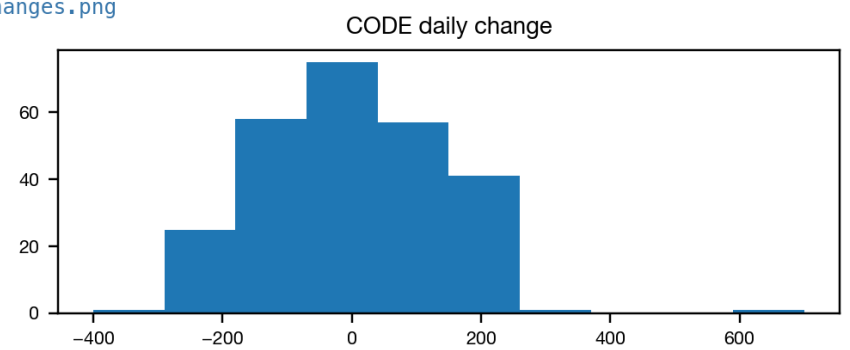
Lab 2 Checks

- The line covers all 260 trading days.
- The histogram counts 259 changes: one fewer than the days.
- Best close 12795 on 2026-08-06. idxmax found the date.
- Charts saved as files, numbers printed as proof.

code_year.png



changes.png



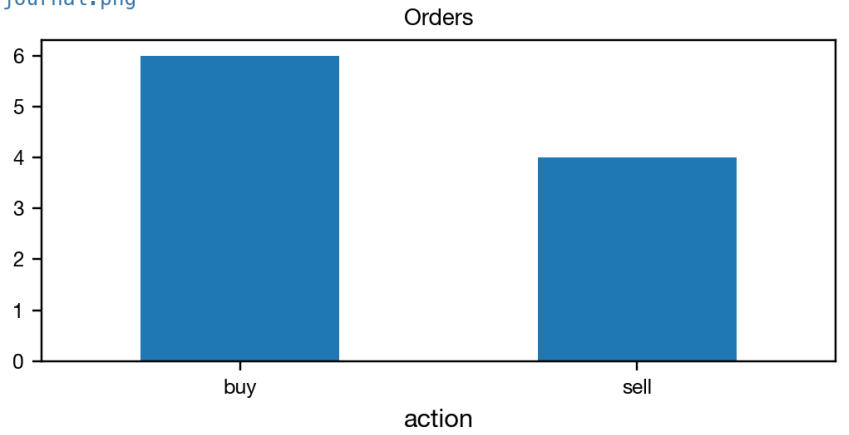
Lab 3 (Take-Home): Journal Chart

- Create journal_chart.py. trades.csv sits beside it.
- Read the journal with pd.read_csv.
- Bar chart the action counts, with a title.
- Print both counts and the total amount traded.

OUTPUT

```
buy 6
sell 4
Traded: 340000
Saved journal.png
```

journal.png



Five Ways to Ruin a Friday Evening

- open() without with: the file stays open, maybe empty.
- "w" on the journal: a week of trades, gone. Append with "a".
- No self in a method's def: takes 1 argument but 2 were given.
- Two charts without figure(): one layered mess.
- Comparing an unstripped line: "sell\n" is not "sell".

> **TIP** strip() every line you read. The newline hides in every comparison you forget.

Exit Quiz and Submission

- 1. Which mode appends without erasing?
- 2. What is self inside a method?
- 3. Which command installs matplotlib?
- 4. Which chart compares category sizes?
- 5. What starts a fresh chart in a script?

TIP Record at least three test cases in comments for each program.

OUTPUT

```
$ ls submit/
journal_by_hand.py   position_report.py
my_charts.py         year_report.py
journal_chart.py is due next session
```

Next Session: Your Position Learns New Tricks

- Inheritance: subclasses extend Position without rewriting it.
- Encapsulation: guard the data inside the object.
- Git begins: versions of your work, before the Week 8 midterm.
- Keep your files, classes and charts. The project half reuses all three.

